

Candidate Sourcing Strategy — Decision Record

Project: AI Talent Lab

Date: June 2026

Status: Yet to decide

Decision in one line: We are pivoting from AI-driven candidate *sourcing* to AI-driven job *distribution + inbound screening*, because reliable, legal, resume-grade sourcing is not achievable for our market — but multi-board posting is, and it gives us real resumes to evaluate.

1. What we originally believed (pre-development)

Our founding hypothesis for the product was an **outbound sourcing engine**:

1. Connect to APIs from LinkedIn, Naukri, and other job boards.
2. Read the JD and automatically pick matching candidates from those sources.
3. Generate an ATS score for each candidate.
4. Send a personalized outreach email containing a magic link.
5. The candidate clicks the link and applies — the same experience as applying from a careers page, except **we came to them**.

The intended differentiation was exactly this reversal: instead of waiting for candidates to apply, **we would source and approach them**. The entire pipeline — JD generation, ATS scoring, magic-link apply, pre-evaluation, interview kit — was built around the assumption that we could obtain candidate profiles, score them, and contact them at scale.

2. What we identified during development

As we built the sourcing layer, every assumption about data access broke down:

Job boards do not give out resumes — only metadata. Across LinkedIn and every major board, the API (where one exists at all) exposes profile metadata: title, company, skills, location. The actual resume is never available programmatically. Resume access is the boards' core revenue, so it is deliberately locked.

Naukri has no developer API. Naukri exposes Resdex as a *solution you subscribe to and use inside their UI*, not a *service you can integrate*. There is no REST API to pull resumes or profiles into our product — at any price tier.

Metadata-only APIs don't fit our market. People Data Labs, Apollo, and similar providers are real and self-serve, but their coverage is strongest for US white-collar tech workers. For Indian candidates — and especially for non-tech and blue-collar roles like civil engineer or data entry — coverage is thin to non-existent. More importantly, **metadata alone cannot tell us whether a candidate is actually skilled**.

Even where profiles exist, they are stale. From our team's discussions with HR practitioners: 40–60% of LinkedIn profiles are unmaintained, and there are consistent gaps between what a profile shows and what

the candidate's real resume contains. Scoring against this data means scoring against an outdated, incomplete picture.

Enrichment vendors are legally radioactive. Proxycurl — a \$10M ARR enrichment provider — was shut down in July 2025 after a LinkedIn lawsuit. France's CNIL fined Kaspr ~\$240,000 in December 2024 for scraping LinkedIn contact data. Building contact-discovery on these vendors means building on legal and continuity risk we cannot absorb as an early-stage company.

Our development data was not real. In dev mode we were generating candidates with an LLM (simulation adapter) — fully fabricated profiles. This meant we could not validate that our sourcing or scoring logic worked against real-world data at all.

3. What we tried

We tested the viable sourcing paths and documented why each failed for our needs:

Tavily web search + LLM dossier extraction. We searched the open web, biased toward profile pages, and used an LLM to extract candidate dossiers. Problems: the search returned no contact email for the vast majority of results (so those candidates were dropped before outreach), the site filters were biased toward tech profiles (GitHub, Stack Overflow, dev.to) and useless for other industries, and the extracted "profile" was a 2–3 sentence snippet — too thin to score meaningfully.

Simulation (LLM-generated profiles). Useful as a dev placeholder, but fabricated. It cannot validate real-world quality and is unsafe for production.

Metadata aggregators (PDL / Apollo). Evaluated and rejected: US-centric, weak in India, no resume depth, and metadata that can't judge real skill.

Email enrichment for missing contacts. Evaluated and rejected: the leading vendor is shut down, the category carries active regulatory penalties, and our own rule already forbids fabricating-and-sending emails.

Conclusion from testing: There is no path to reliable, legal, resume-grade candidate sourcing for an India-first, all-industries product. This is a structural constraint of the market, not a gap in our implementation.

4. The solution we are adopting

We invert the model: **stop sourcing candidates, start distributing jobs and capturing inbound applications.**

The flow becomes:

1. Hire request → approval → JD generation → bias check → JD approval → **position opened** (*unchanged — already built*).
2. On opening, the position is **automatically posted to job boards** the customer has selected.
3. Candidates discover the role and **apply through our magic-link / careers flow**, submitting a **real, current resume**.
4. Each application is **ATS-scored on the real resume** (not metadata), deduplicated against our internal pool, and ranked.

5. Above-threshold candidates flow into **pre-evaluation** → **interview kit** → **decision**, with LLM-generated summaries for the hiring team (*unchanged* — *already built*).

Distribution channels, by priority:

- **Google for Jobs — free, first.** Google crawls any careers page carrying **JobPosting** structured data and surfaces it in search. No API, no partnership, no cost, works for every industry. Highest ROI action.
- **Indeed — XML feed.** Standard, well-documented; free organic indexing with optional paid boost.
- **LinkedIn and others — partner/feed, later.**
- **Naukri and niche/industry-specific boards — per-client.** No clean public posting API, so we offer the cleanly-integrable boards by default and build specific boards on customer demand.

5. Why we are choosing this now

It is the only side of the market that is actually open. Boards guard resumes (their revenue) but want job postings (their inventory). Posting is structurally available; sourcing is structurally closed. We are moving to the open side of every board.

It fixes our biggest data-quality problem. Candidates now arrive with real, current resumes through our own apply flow. The two-step ATS scoring engine we already built finally runs on real data — its quality improves overnight with no change to the scoring code.

It is legal and durable. No scraping, no ToS violations, no dependency on vendors that can be sued out of existence. Important for a company with zero customers and no margin for legal risk.

It is industry-agnostic. Civil engineer, data entry, sales, nursing — anyone who can see a job posting can apply. The product no longer breaks on non-tech roles.

It fits our market and stage. India-first SMB customers, low cost (Google for Jobs is free), and a clear value proposition: *generate the JD, post everywhere, and only ever review a ranked, pre-screened shortlist.*

It relocates our moat to something defensible. We no longer compete on sourcing. We compete on the post-application AI pipeline — instant resume-based ATS scoring, automated pre-evaluation, interview kits, and the full approval workflow — plus a compounding, consented, resume-backed **talent pool** of past applicants that becomes reusable across each customer's future roles. No competitor has *our* pool.

6. Impact on the existing codebase

Unchanged / now more central: JD generation pipeline, bias check, hire-request approval chain, magic-link apply, careers page, ATS scoring, pre-evaluation, interview kit, deduplication, talent pool.

Repurposed: The outreach-email + magic-link system shifts from *cold outreach to sourced strangers* → *warm re-engagement of our consented talent pool*. Same code, compliant use.

Deprecated / removed from the core flow: Tavily adapter, simulation adapter, PDL/Apollo/enrichment adapters, the scheduled auto-sourcing loop, and the dead Proxycurl config.

New to build: A job-distribution module — a **JobBoardAdapter** interface mirroring our existing clean adapter pattern (`post()`, `update()`, `close()`), starting with Google for Jobs and Indeed; a "where to

post" selector in the UI; and inbound source attribution (which board drove which application).

7. Known risk and mitigation

Inbound is **pull, not push**. High-volume and common roles (data entry, sales) will draw applicants readily. Rare or niche roles (e.g., a senior civil engineer at an unknown SMB) may draw few — and we can no longer proactively go get them. **Mitigations:** paid/sponsored posts for hard-to-fill roles, and talent-pool re-engagement once the pool is populated. We will set this expectation with customers up front so low inbound on rare roles is understood as a market reality, not a product defect.

Appendix — References

- Pin — Profile Data APIs for Recruiting, 16 Providers Compared (2026): Proxycurl shutdown, GDPR/CNIL fines, coverage analysis — <https://www.pin.com/blog/profile-data-apis/>
- Naukri Resdex — Resume Database Access (product, no public API) — <https://www.naukri.com/recruit/resume-database-access-resdex>
- Indeed Smart Sourcing — <https://www.indeed.com/employers/smart-sourcing>
- Google — JobPosting structured data documentation — <https://developers.google.com/search/docs/appearance/structured-data/job-posting>
- Joveo — Google for Jobs: The Ultimate Guide — <https://www.joveo.com/google-for-jobs-the-ultimate-guide/>
- ClarityHire — How an Indeed XML Feed and Google for Jobs JSON-LD Work — <https://www.clarity-hire.com/blog/indeed-google-jobs-xml-feed-explained>